

# Apache Spark con Python

Preguntas que pueden salir durante o después de la clase, ordenadas por dificultad, con la respuesta corta lista. No hay que memorizarlas: léelas dos veces y quedate con la idea central de cada una. Las respuestas están escritas para decirse en voz alta en menos de treinta segundos.

## Nivel 1 - Lo básico

*Casi seguro sale alguna de estas. Respuesta corta y segura vale más que una larga.*

### 1. Qué es Apache Spark en una frase?

Un motor que divide un trabajo grande de datos en pedazos pequeños y los procesa al mismo tiempo en varios computadores (o en varios núcleos de uno solo).

### 2. Qué diferencia hay entre Spark y PySpark?

Spark es el motor, escrito en Scala y corriendo sobre Java. PySpark es la librería de Python que le da órdenes a ese motor. **pip install pyspark** descarga los dos: el paquete trae el motor empaquetado adentro.

### 3. Hay que descargar Spark aparte, desde la página de Apache?

No. Solo **pip install pyspark**. Lo único que se instala por separado es Java, porque el motor corre sobre la JVM. En Google Colab, Java ya viene puesto.

### 4. Qué es una SparkSession?

La puerta de entrada: el objeto que representa tu conexión con el motor. Sin ella no se puede hacer nada, y por eso siempre es la primera línea del código.

### 5. Qué significa local[\*]?

'Simula el clúster aquí mismo, en esta máquina, usando todos los núcleos disponibles.' local usa 1 núcleo, local[4] usa 4, local[\*] usa todos.

### 6. Necesito un clúster para aprender Spark?

No. Con local[\*] corre en tu portátil y el código es exactamente el mismo que en producción. Lo único que cambia es esa línea.

### 7. Qué es un DataFrame en Spark?

Una tabla distribuida entre varias máquinas, donde cada columna tiene nombre y tipo. Es la estructura con la que se trabaja el 100% del tiempo.

### 8. Para que sirve printSchema()?

Muestra el nombre y el tipo de cada columna. Es el equivalente de df.dtypes en pandas y debería ser lo primero que revisas después de leer un archivo.

## Nivel 2 - Como funciona por dentro

*Aquí se separa quien vio un tutorial de quien entendió la herramienta.*

### 9. Qué es el driver y que es un executor?

El **driver** es tu script de Python: arma el plan y reparte tareas, pero no procesa datos. Los **executors** son los trabajadores que sí procesan. El paralelismo vive en los executors.

### 10. Qué es una partición?

Cada pedazo en el que Spark corta el archivo. Es la unidad de paralelismo: una partición equivale a una tarea, y cada executor procesa las suyas al mismo tiempo que los demás.

### 11.Cuál es la diferencia entre una transformación y una acción?

Las **transformaciones** (select, filter, withColumn, groupBy, join) no ejecutan nada: solo van anotando pasos en un plan. Las **acciones** (show, count, collect, write, toPandas) disparan la ejecución de todo el plan acumulado.

### 12. Qué es la evaluación perezosa y para qué sirve?

Que Spark espera a tener el plan completo antes de ejecutar. Sirve porque al ver la receta entera puede reordenarla para hacer menos trabajo: si al final pides 2 columnas, nunca lee las otras 30 del disco.

### 13. Qué es Catalyst?

El optimizador de Spark. Es el que toma el plan que armaste y lo reescribe en una versión más eficiente antes de ejecutarlo. Es la razón por la que la pereza vale la pena.

### 14. Qué es un shuffle y por qué importa?

Es mover datos entre executors para juntar filas que van al mismo grupo. Pasa en groupBy y en join. Viaja por la red y es **la operación más costosa que existe en Spark**: es la causa número uno de que un proceso se demore.

### 15. Cómo se depura un proceso que va lento?

Con **.explain()**, que imprime el plan de ejecución, y con el panel web en **localhost:4040**, que muestra los tiempos por etapa y las particiones. El culpable suele ser un shuffle.

### 16. Por qué Spark es rápido si Python es lento?

Porque el motor está en Scala sobre la JVM. Python solo manda instrucciones; los datos nunca pasan por Python. La excepción son las UDFs, y por eso se evitan.

### 17. Qué es un RDD?

La API original de bajo nivel de Spark. Hoy se usa DataFrame porque pasa por Catalyst y rinde mejor. El RDD sigue existiendo por debajo, pero no se escribe directamente.

### 18. Qué es Parquet y por que se prefiere sobre CSV?

Un formato columnar y comprimido. Conserva los tipos de dato, pesa 5-10 veces menos que CSV, y puede leer solo las columnas que pidas sin tocar el resto del archivo. Es el estándar del data lake.

## Nivel 3 - pandas vs Spark

*La comparación que siempre sale. Ojo con la trampa de la primera pregunta.*

### 19. Spark reemplaza a pandas?

No, conviven. El flujo real es: Spark procesa los 500 GB y devuelve un resumen de 2 MB; pandas grafica ese resumen. **pandas para explorar, Spark para producir.**

### 20. Cuándo usar pandas y cuando Spark?

Regla informal: menos de ~5 GB, pandas. Mas que eso, o si el proceso va a correr todas las noches en producción, Spark.

### 21. Spark siempre es más rápido que pandas?

**No, y esto es clave:** con datos pequeños Spark es más LENTO. Levantar la máquina virtual de Java tarda segundos siempre, y ese costo fijo solo se amortiza con volumen.

### 22. Qué cosas de pandas NO existen en Spark?

El índice (nada de `.loc` ni `.iloc`), `inplace=True`, `iterrows()`, graficar directamente, `.interpolate()` y `.resample()`. Todo lo que asume que los datos están juntos y en orden en una sola máquina.

### 23. Por qué Spark no tiene índice?

Porque las filas están repartidas entre varias máquinas y no hay un orden natural garantizado a menos que llames `orderBy` explícitamente. Pedir 'la fila 5' no tiene sentido.

### 24. Qué significa que los DataFrames de Spark son inmutables?

Que ninguna operación los modifica: todas devuelven un DataFrame nuevo. Por eso siempre hay que reasignar a una variable y por eso no existe `inplace=True`.

### 25. Cómo se pasa de Spark a pandas?

Con `.toPandas()`. Pero solo sobre resultados ya agregados y pequeños, porque carga todo en la RAM de una sola máquina. Es el último paso del proceso, nunca el primero.

### 26. Existe forma de escribir sintaxis de pandas y que corra distribuido?

Si, el módulo `pyspark.pandas`. Escribes como en pandas (con índice incluido) y por debajo ejecuta en Spark. No cubre absolutamente todo, pero ayuda bastante en la transición.

## Nivel 4 - Código y decisiones técnicas

*Preguntas sobre lo que se ve en pantalla durante la demostración.*

### 27. Por qué declaraste el esquema a mano en vez de usar inferSchema?

Porque `inferSchema` obliga a Spark a leer el archivo dos veces: una para adivinar los tipos y otra para cargarlo. Con 5.000 filas no se nota, con 500 GB pagas doble lectura. Además adivinar sale mal: un código como 00123 lo convierte en número y le borra los ceros.

### 28. Por qué imputaste los nulos por grupo y no con el promedio general?

Porque los grupos se comportan distinto. En el ejemplo, Cálculo promedia 2.9 e Inglés 4.3; rellenar una nota faltante de Cálculo con el promedio general (3.7) le regala casi un punto a ese estudiante y contamina el resultado del curso.

### 29. Qué hace Window.partitionBy?

Agrupar por una columna **sin colapsar las filas**. Un groupBy convierte 5.000 filas en 5; una ventana deja las 5.000 filas intactas pero cada una puede consultar un cálculo hecho sobre su propio grupo. Es el equivalente de groupby().transform() en pandas.

### 30. Qué hace F.coalesce?

Viene de SQL y significa 'devuélveme el primer valor que no sea nulo'. Se usa para rellenar: si el dato existe lo deja, si es nulo mete el valor de respaldo.

### 31. Por qué el dropDuplicates va antes de calcular el promedio?

Porque si va después, las filas duplicadas pesan doble en el cálculo y sesgan el promedio. El orden de las operaciones de limpieza importa.

### 32. Qué es una UDF y por qué se evita?

Una función de Python que se aplica fila por fila. Se evita porque el motor está en Java: cada fila tendría que cruzar de Java a Python y volver, y el rendimiento se cae al piso. Siempre que exista una función de pyspark.sql.functions, se usa esa.

### 33. Escribir en SQL rinde distinto que usar la API de DataFrames?

No. Las dos pasan por el mismo optimizador (Catalyst) y rinden igual. Es cuestión de gusto.

### 34. Qué hace F.when().otherwise()?

Es el CASE WHEN de SQL: evalúa una condición y devuelve un valor u otro. Es como se crean columnas categóricas o segmentos.

### 35. Por qué la carpeta de salida tiene varios archivos part-00000 en vez de uno?

Porque cada partición se escribe por separado. Es el comportamiento normal de un sistema distribuido, y al leer la carpeta Spark vuelve a juntar todo automáticamente.

### 36. Cómo demuestras que el optimizador realmente funciona?

Con .explain(): en la línea FileScan se ve que Spark solo va a leer las columnas que la consulta necesita, no todas las del archivo. Nadie se lo pidió; lo dedujo porque vio el plan completo antes de ejecutar.

## Nivel 5 - Contexto e ingeniería de datos

*Preguntas de alrededor, para ubicar Spark en el panorama.*

### 37. Dónde encaja Spark en un pipeline de datos?

En la etapa de procesamiento. Los datos nacen en bases transaccionales o APIs, se ingestan al data lake, Spark los limpia y transforma, y de ahí salen a dashboards, reportes o modelos.

### 38. Qué diferencia hay entre ETL y ELT?

ETL transforma antes de guardar; ELT guarda crudo y transforma después. Hoy domina el ELT porque el almacenamiento salió barato, y Spark es la herramienta típica de la T.

### 39. Qué son las capas bronze, silver y gold?

Cómo se organiza el data lake: bronze es el dato crudo tal como llegó, silver el dato limpio y validado, gold el dato agregado y listo para el negocio.

### 40. Spark compete con Airflow?

No, se complementan. Airflow es un orquestador: decide **cuándo** corre cada proceso. Spark decide **cómo** se procesa.

### 41. Dónde se usa Spark en la vida real?

En Databricks, AWS EMR, Google Dataproc y Azure Synapse. Databricks es la plataforma de la empresa que fundaron los creadores de Spark, y tiene una edición Community gratis.

### 42. Spark sirve para machine learning?

Sí, tiene un módulo llamado MLlib con algoritmos que corren distribuidos. También sirve para streaming con Structured Streaming. Es la ventaja de tener un solo motor.

## Y si algo sale mal

| Situación                              | Qué hacer                                                                                                                                |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Si te preguntan algo que no sabes      | La respuesta correcta es: ' <b>No lo sé con certeza, pero mi intuición es que...</b> ' Inventar es lo único que sí hunde una exposición. |
| Si la demostración falla en vivo       | Dilo sin drama: 'se me cayó el entorno, vamos por las diapositivas', y sigues. Lo que se nota mal no es que falle, es que te desarmes.   |
| Si te piden resumir Spark en una frase | 'pandas para explorar; Spark para cuando el dato ya no cabe en una máquina.'                                                             |